

iOS 面试题代码实现详解

1. 循环引用 & 解决方案

大白话：

A 拿着 B 的地址，B 也拿着 A 的地址，谁也不放手，内存就泄露了。

代码实现：

Code block

```
1 // Person.h
2 #import <Foundation/Foundation.h>
3
4 @interface Person : NSObject
5 @property (nonatomic, strong) NSString *name;
6 @property (nonatomic, strong) Person *friend;
7
8 - (instancetype)initWithName:(NSString *)name;
9 @end
```

Code block

```
1 // Person.m
2 #import "Person.h"
3
4 @implementation Person
5
6 - (instancetype)initWithName:(NSString *)name {
7     if (self = [super init]) {
8         _name = name;
9     }
10    return self;
11 }
12
13 - (void)dealloc {
14     NSLog(@"Person %@ is being deallocated.", self.name);
15 }
16
17 @end
```

Code block

```
1 // 在 ViewController 里演示循环引用
2 - (void)viewDidLoad {
3     [super viewDidLoad];
4
5     Person *tom = [[Person alloc] initWithName:@"Tom"];
6     Person *jerry = [[Person alloc] initWithName:@"Jerry"];
7
8     tom.friend = jerry;
9     jerry.friend = tom;
10 }
```

解决方案：使用 weak

Code block

```
1 // 修改 Person.h
2 @interface Person : NSObject
3 @property (nonatomic, strong) NSString *name;
4 @property (nonatomic, weak) Person *friend;
5
6 - (instancetype)initWithName:(NSString *)name;
7 @end
```

2. Block & 循环引用

大白话：

Block 里用 self，self 里又持有 Block，形成循环引用。

代码实现：

Code block

```
1 // NetworkManager
2 @interface NetworkManager : NSObject
3 - (void)fetchDataWithCompletion:(void(^)(NSData *data, NSError
4 *error))completion;
5 @end
6
7 @implementation NetworkManager
8 - (void)fetchDataWithCompletion:(void(^)(NSData *data, NSError
9 *error))completion {
```

```

8      dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(2.0 *
NSEC_PER_SEC)), dispatch_get_main_queue(), ^{
9          if (completion) {
10              completion(nil, nil);
11          }
12      });
13  }
14  @end

```

Code block

```

1  // ViewController
2  @interface ViewController ()
3  @property (nonatomic, strong) NetworkManager *networkManager;
4  @end
5
6  @implementation ViewController
7
8  - (void)viewDidLoad {
9      [super viewDidLoad];
10     self.networkManager = [[NetworkManager alloc] init];
11
12     // 错误示范: 循环引用
13     [self.networkManager fetchDataWithCompletion:^(NSData *data, NSError
*error) {
14         self.view.backgroundColor = [UIColor redColor];
15     }]];
16 }
17
18 @end

```

解决方案: __weak + __strong

Code block

```

1  - (void)viewDidLoad {
2      [super viewDidLoad];
3      self.networkManager = [[NetworkManager alloc] init];
4
5      __weak typeof(self) weakSelf = self;
6
7      [self.networkManager fetchDataWithCompletion:^(NSData *data, NSError
*error) {
8          __strong typeof(weakSelf) strongSelf = weakSelf;
9          if (!strongSelf) return;
10

```

```
11         strongSelf.view.backgroundColor = [UIColor greenColor];
12     }];
13 }
```

3. Category 和 Extension

大白话:

Category 是给别人家的书写笔记, Extension 是给自己家的书写私密批注。

Category 代码:

Code block

```
1 // NSString+Utils.h
2 #import <Foundation/Foundation.h>
3
4 @interface NSString (Utils)
5 - (BOOL)isValidEmail;
6 - (NSString *)reversedString;
7 @end
```

Code block

```
1 // NSString+Utils.m
2 #import "NSString+Utils.h"
3
4 @implementation NSString (Utils)
5
6 - (BOOL)isValidEmail {
7     NSString *pattern = @"[A-Z0-9a-z._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,4}";
8     NSPredicate *predicate = [NSPredicate predicateWithFormat:@"SELF MATCHES
9 %@", pattern];
10     return [predicate evaluateWithObject:self];
11 }
12
13 - (NSString *)reversedString {
14     NSMutableString *reversed = [[NSMutableString alloc] init];
15     for (NSInteger i = self.length - 1; i >= 0; i--) {
16         [reversed appendFormat:@"%C", [self characterAtIndex:i]];
17     }
18     return [reversed copy];
19 }
20 @end
```

Extension 代码:

Code block

```
1 // Person.h
2 @interface Person : NSObject
3 @property (nonatomic, strong) NSString *name;
4 @end
```

Code block

```
1 // Person.m
2 #import "Person.h"
3
4 @interface Person ()
5 @property (nonatomic, strong) NSString *address;
6 - (void)privateMethod;
7 @end
8
9 @implementation Person
10
11 - (void)privateMethod {
12     NSLog(@"This is a private method.");
13 }
14
15 - (void)dealloc {
16     NSLog(@"Person %@ is being deallocated.", self.name);
17 }
18
19 @end
```

4. UITableView 的 Cell 复用

大白话:

就像公交车座位，上来一个乘客坐下，下去一个乘客让座，只复用不重复创建。

代码实现:

Code block

```
1 - (void)viewDidLoad {
2     [super viewDidLoad];
```

```

3     UITableView *tableView = [[UITableView alloc]
initWithFrame:self.view.bounds];
4     tableView.dataSource = self;
5     [self.view addSubview:tableView];
6 }
7
8 #pragma mark - UITableViewDataSource
9 - (NSInteger)tableView:(UITableView *)tableView numberOfRowsInSection:
(NSInteger)section {
10     return 100;
11 }
12
13 - (UITableViewCell *)tableView:(UITableView *)tableView cellForRowAtIndexPath:
(NSIndexPath *)indexPath {
14     static NSString *cellIdentifier = @"MyCellIdentifier";
15
16     UITableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:cellIdentifier];
17
18     if (cell == nil) {
19         cell = [[UITableViewCell alloc]
initWithStyle:UITableViewCellStyleDefault reuseIdentifier:cellIdentifier];
20         NSLog(@"Creating new cell for row %ld", (long)indexPath.row);
21     }
22
23     cell.textLabel.text = [NSString stringWithFormat:@"Item %ld",
(long)indexPath.row];
24     return cell;
25 }

```

5. KVO (键值观察)

大白话:

我盯着你，你一动我就收到通知。

代码实现:

Code block

```

1 // Person.h
2 @interface Person : NSObject
3 @property (nonatomic, strong) NSString *name;
4 @property (nonatomic, assign) NSInteger age;
5 @end
6

```

```
7 // Person.m
8 @implementation Person
9 @end
```

Code block

```
1 // ViewController
2 @interface ViewController ()
3 @property (nonatomic, strong) Person *person;
4 @end
5
6 @implementation ViewController
7
8 - (void)viewDidLoad {
9     [super viewDidLoad];
10    self.person = [[Person alloc] init];
11    self.person.name = @"Old Name";
12
13    [self.person addObserver:self
14                     forKeyPath:@"name"
15                     options:NSKeyValueObservingOptionNew |
16                           NSKeyValueObservingOptionOld
17                     context:nil];
18
19    - (void)observeValueForKeyPath:(NSString *)keyPath
20                               ofObject:(id)object
21                               change:(NSDictionary *)change
22                               context:(void *)context {
23        if ([keyPath isEqualToString:@"name"]) {
24            NSLog(@"Name changed from %@ to %@", change[NSKeyValueChangeOldKey],
25            change[NSKeyValueChangeNewKey]);
26        }
27
28    - (void)dealloc {
29        [self.person removeObserver:self forKeyPath:@"name"];
30    }
31
32 @end
```

6. GCD (Grand Central Dispatch)

大白话:

把任务交给系统，系统帮你找工人（线程）去做。

代码实现：

Code block

```
1  - (void)viewDidLoad {
2      [super viewDidLoad];
3
4      // 后台执行耗时任务
5      dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT,
6          0), ^{
7
8          NSData *data = [self fetchBigDataFromNetwork];
9
10         // 回到主线程更新 UI
11         dispatch_async(dispatch_get_main_queue(), ^{
12             self.imageView.image = [UIImage imageWithData:data];
13         });
14     });
15
16     // 串行队列
17     dispatch_queue_t serialQueue =
18     dispatch_queue_create("com.myapp.serialqueue", DISPATCH_QUEUE_SERIAL);
19     dispatch_async(serialQueue, ^{
20         NSLog(@"Task 1 on serial queue");
21     });
22     dispatch_async(serialQueue, ^{
23         NSLog(@"Task 2 on serial queue");
24     });
25 }
26
27 - (NSData *)fetchBigDataFromNetwork {
28     sleep(2);
29     return [NSData data];
30 }
```

(注：文档部分内容可能由 AI 生成)